

Boot Image Pipeline — From Block Design to Power-On

This guide explains **how the files in `release/` are made and why the board boots by itself**: how a Vivado block design becomes a bitstream, how a bootloader ELF gets stitched into that bitstream without re-running synthesis, how everything is laid out in the QSPI flash, and what actually happens at power-on. It is the "behind the scenes" companion to the [Standalone Boot Mode guide](#).

You do not need any of this to *use* the board. Read it when you want to know how the pieces fit, or when you need to rebuild the boot images yourself.

The one key idea

A bitstream does not only describe logic and routing — **it also contains the initial contents of every block RAM on the chip**. The CPU's 128 KB local memory is built from BRAM, so whatever program those initialization frames hold is simply *there* the moment the FPGA finishes configuring, before any software has run. That is:

- why a bootloader can be "baked into" the hardware image,
- why it is restored on every power-on and can never be bricked,
- and why a program can be swapped into an already-built bitstream in seconds (`updatemem`) — only BRAM init frames change; logic, routing, and timing are untouched.

Pipeline overview

```
block design
| synthesis + implementation (Vivado, one-time)
▼
top.bit ..... logic + routing + BLANK BRAM
|
| + top_wrapper.mmi (address→BRAM map, in the XSA)
| + bootloader ELF
▼ updatemem
boot.bit / boot_srec.bit ..... BRAM init = bootloader
|
| + application image (raw binary / SREC)
▼ write_cfgmem
*.mcs ..... flash-shaped image
|
▼ program_hw_cfgmem
QSPI flash
|
▼ power-on
FPGA configures itself → bootloader in BRAM → app runs
```

Stage 1 — Block design → `top.bit` (Vivado, one-time)

Synthesis and implementation turn the block design into a stream of *configuration frames*: every LUT, every route, and the INIT values of every RAMB36 primitive. At this point the BRAM init values are all zeros — `top.bit` programmed on its own leaves the CPU executing empty memory (silent UART, no signs of life). `top.bit` is deliberately kept in this "blank" state — the bootloader is stitched in afterwards (Stage 3).

Vivado can also bake an ELF in at `write_bitstream` time (ELF association). This repo does not: keeping `top.bit` generic means a bootloader change never requires re-running the implementation.

Stage 2 — `top_wrapper.mmi`: the address-to-BRAM map (Vivado, one-time)

The 128 KB local memory is not one physical block — it is 32 RAMB36 primitives (4 KB each) scattered across the die, each holding only a few bits of every 32-bit word. Which physical cell stores the byte at CPU address `0x1234` is only known **after placement**, so Vivado writes this mapping into `top_wrapper.mmi` (shipped inside `release/top_wrapper.xsa`). Any hardware rebuild changes the placement, so the MMI must always be re-extracted from the matching XSA.

Stage 3 — `updatemem`: stitch an ELF into the bitstream (seconds, no re-synthesis)

```
updatemem -force -meminfo top_wrapper.mmi -data <bootloader>.elf \
-bit release/top.bit -proc top_i/microblaze_riscv_0 -out <boot>.bit
```

`updatemem` reads the ELF's loadable segments (the bootloader lives at `0x0-0x8000`), uses the MMI to scatter each byte into the right INIT frame of the right RAMB36, and rewrites **only those frames**. Logic and timing are bit-for-bit unchanged, which is why this takes ~6 seconds instead of a 20-minute implementation run. `release/boot_srec.bit` is exactly `top.bit` with the bootloader ELF stitched in.

Stage 4 — `write_cfgmem`: lay out the flash image (Vivado)

The FPGA's configuration engine expects the bitstream to sit at flash offset `0x0` in a specific byte order. `write_cfgmem` produces that image, and can place additional raw data at other offsets in the same file:

```
write_cfgmem -force -format mcs -size 4 -interface SPIx4 \
-loadbit "up 0x0 boot_srec.bit" \
-loaddata "up 0x00340000 showcase.srec" \
-file official_boot.mcs
```

Flash map (4 MB QSPI)

Offset	Contents	Read by
--------	----------	---------

Offset	Contents	Read by
0x000000–0x21FFFF	Bitstream (with bootloader in BRAM init)	FPGA configuration engine, at power-on
0x340000	Application slot (SREC text)	srec_spi_boot loader

The gap between the two regions is unused headroom; the slot offset is a bootloader configuration constant (`blconfig.h`) and can be moved if an application ever outgrows it.

Stage 5 — `program_hw_cfgmem`: write the flash over JTAG (Vivado Hardware Manager)

The FPGA has no direct "write my flash" JTAG command, so Vivado first programs a temporary *bridge* bitstream that turns the FPGA into a JTAG ↔ SPI programmer, then erases, writes, and verifies. The important property setting:

```
set_property PROGRAM.ADDRESS_RANGE {use_file} $cfg
```

`use_file` scopes the erase+program to only the address ranges present in the MCS — that is what makes it safe to update one slot (or the bitstream) without touching the others. Afterward, `boot_hw_device` forces the FPGA to reconfigure from flash — functionally identical to a power-cycle, handy for verifying the real boot path without touching the board.

Stage 6 — Power-on: one flash, two readers

The same QSPI flash is read twice, by two different "readers":

- 1. **The configuration engine** — dedicated silicon that exists before your design does. With the mode pins set to Master-SPI it streams the bitstream from offset `0x0` into the chip. The moment the BRAM init frames load, the bootloader *appears* in memory — no software was involved, which is why the bootloader survives anything and is restored on every power-on.
- 2. **The design's own QSPI flash controller** — once the design is alive, the CPU leaves reset at address `0x0` , runs the bootloader, and the bootloader uses this controller to read the *application* from its flash slot, copy it to SRAM, and jump to it.

The block design therefore provides the container (BRAM) and the mover (flash controller); `updatemem` decides what the container holds at power-on, and the flash layout decides what gets moved.

Which tool does what

Step	Tool	Vivado needed?
Build bootloader / application ELFs	Vitis	No
<code>objcopy -O srec (app → SREC)</code>	RISC-V toolchain (ships with Vitis)	No

Step	Tool	Vivado needed?
Stitch ELF into bitstream (<code>updatemem</code>)	CLI tool — ships with both Vitis and Vivado	No
JTAG load / debug (<code>xsdb</code> , IDE)	Vitis	No
Synthesize / implement → <code>top.bit</code> + XSA/MMI	Vivado	Yes (one-time)
Build flash image (<code>write_cfgmem</code>)	Vivado Tcl	Yes
Program flash, range-scoped (<code>program_hw_cfgmem</code> + <code>use_file</code>)	Vivado Hardware Manager	Yes

In short: **all software work is Vitis-only**, and once the hardware is frozen even the ELF-into-bitstream step needs nothing from Vivado. Vivado remains necessary for the one-time hardware build and for the flash-image/programming steps — this repo's verified scripts drive it in batch mode, so it never has to be opened interactively.

Vitis also ships a standalone `program_flash` CLI that can write an image to a given flash offset through the FPGA. It has not been validated on this board; the flows in this repo use the Vivado Hardware Manager path above, which is board-verified.

The prebuilt artifacts in `release/`

File	What it is
<code>top.bit</code>	Implementation output, blank BRAM — never boot this alone
<code>top_wrapper.xsa</code>	Hardware handoff for Vitis; contains <code>top_wrapper.mmi</code>
<code>boot_srec.bit</code>	<code>top.bit</code> + SREC bootloader (JPROGRAM = remote power-cycle)
<code>official_boot.mcs</code>	Full flash image: bitstream + bootloader + demo app SREC at <code>0x340000</code>